

聚 类



时间：2025年5月

TRANSWARP
星 环 科 技



目录 >

CONTENTS

- 1 Kmeans 聚类
- 2 层次聚类
- 3 Dbscan 密度聚类

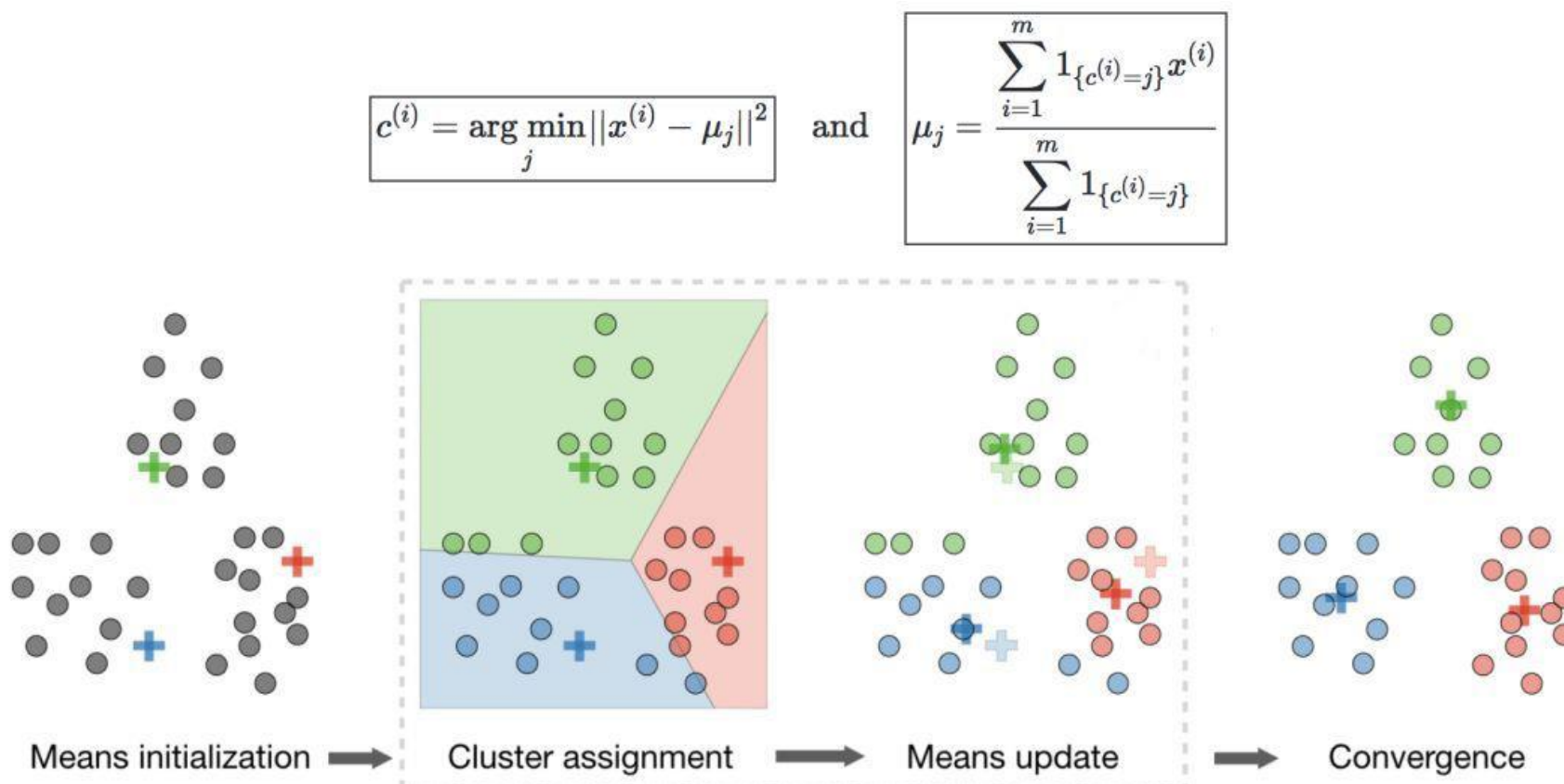


1 chapter

Kmeans 聚类

K-means 算法理解

- **K**: 初始中心点**个数** (计划聚**类数**)
- **means**: 中心点时每个簇的所有样本的**平均值**

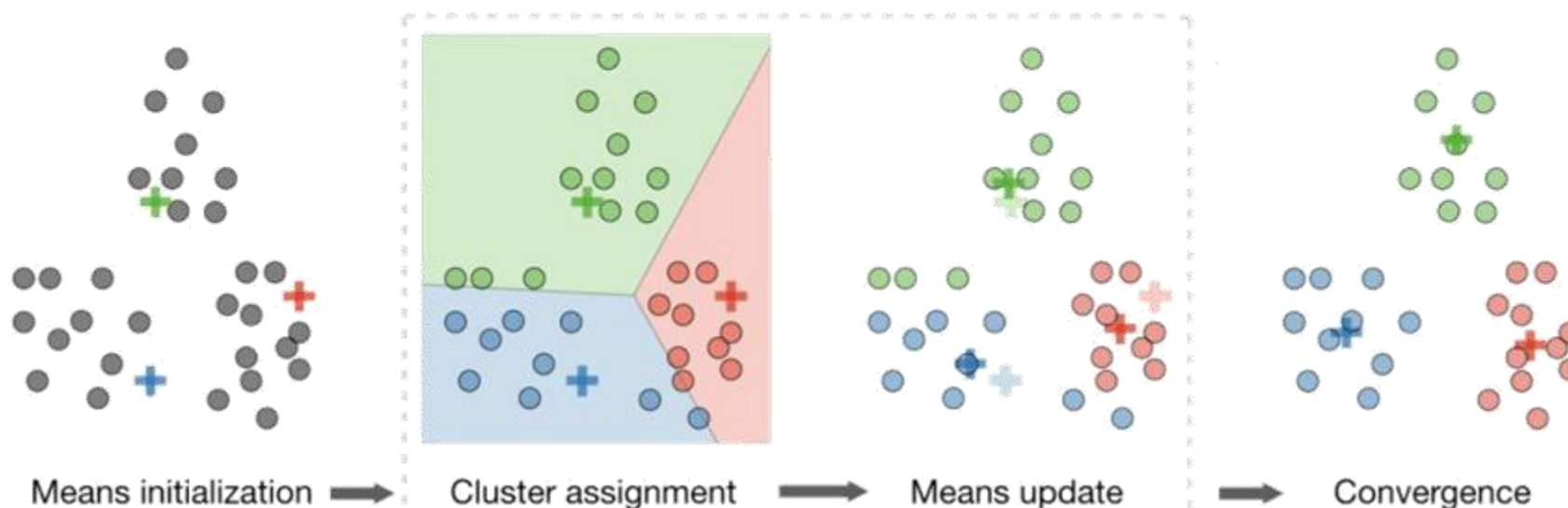


K-means 算法流程

- 初始化**K**个初始中心点
- E步：计算**每个点**到K个中心点的**距离**，**离谁近就给谁**（打标签）
- M步：计算每个簇的**均值点**作为新的中心点
- 不断迭代**EM**步，直到中心点**不在移动**

$$c^{(i)} = \arg \min_j \|x^{(i)} - \mu_j\|^2 \quad \text{and}$$

$$\mu_j = \frac{\sum_{i=1}^m 1_{\{c^{(i)}=j\}} x^{(i)}}{\sum_{i=1}^m 1_{\{c^{(i)}=j\}}}$$



K-means 算法流程



K-means 算法与 EM 算法

■ 监督学习求解：极大似然估计

- 已知 X 和 Y ，直接 $\operatorname{argmax} P(Y|X;\theta)$ 计算出模型参数 θ

■ 非监督学习求解：EM算法

- 已知 X , Y 未知，求模型参数 θ
- **E步**：假设模型参数 θ ，根据 $Y=f(X;\theta)$ 计算出 Y (分布)--->**给参数得到Y**
- **M步**：有了 X 和 Y ，使用 $\operatorname{argmax} P(Y|X;\theta)$ 计算出模型参数 θ ---->**有了Y，算参数**

K-means 计算案例

- 1、随机设置**K个**特征空间内的点作为**初始的聚类中心**（本案例中设置p1和p2）

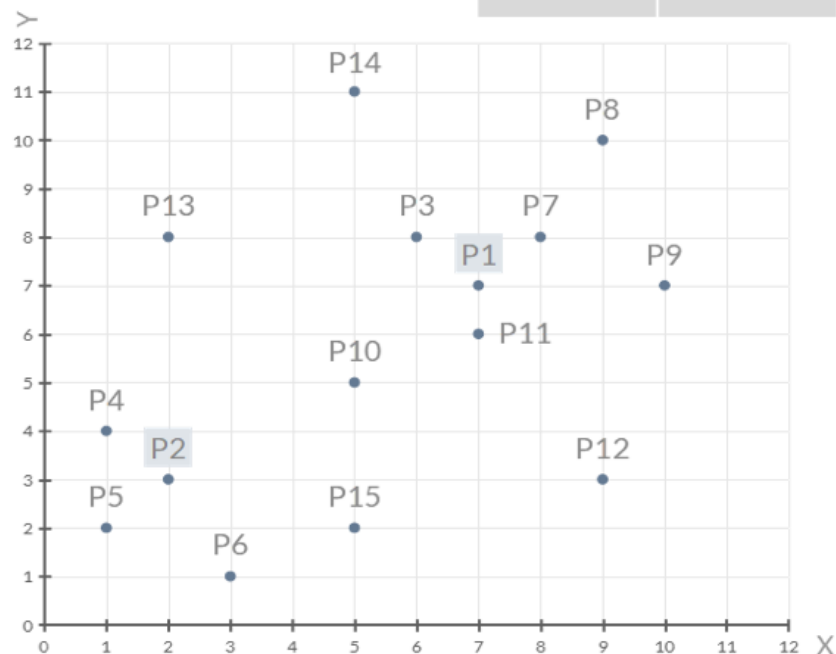
Euclidean

$$\sqrt{\sum_{i=1}^k (x_i - y_i)^2}$$

$$d = \sqrt{(x_n - x_1)^2 + (y_n - y_1)^2}$$

	X 值	Y 值
P1	7	7
P2	2	3
P3	6	8
P4	1	4
P5	1	2
P6	3	1
P7	8	8

	X 值	Y 值
P8	9	10
P9	10	7
P10	5	5
P11	7	6
P12	9	3
P13	2	8
P14	5	11
P15	5	2



K-means 计算案例

- 2、对于其他每个点计算到K个中心的距离，未知的点选择最近的一个聚类中心点作为标记类别

P1	P3	P7	P8	P9	P10	P11	P12	P14
P2	P4	P5	P6	P13	P15			

	P1 (7,7)	P2 (2,3)
P3	1.41	6.40
P4	6.71	1.41
P5	7.81	1.41
P6	7.21	2.24
P7	1.41	7.81
P8	3.61	9.90

	P1 (7,7)	P2 (2,3)
P9	3	8.94
P10	2.83	3.61
P11	1	5.83
P12	4.47	7.00
P13	5.10	5.00
P14	4.47	8.54
P15	5.39	3.16

K-means 计算案例

- 3、接着对着标记的聚类中心之后，重新计算出每个聚类的新中心点（平均值）

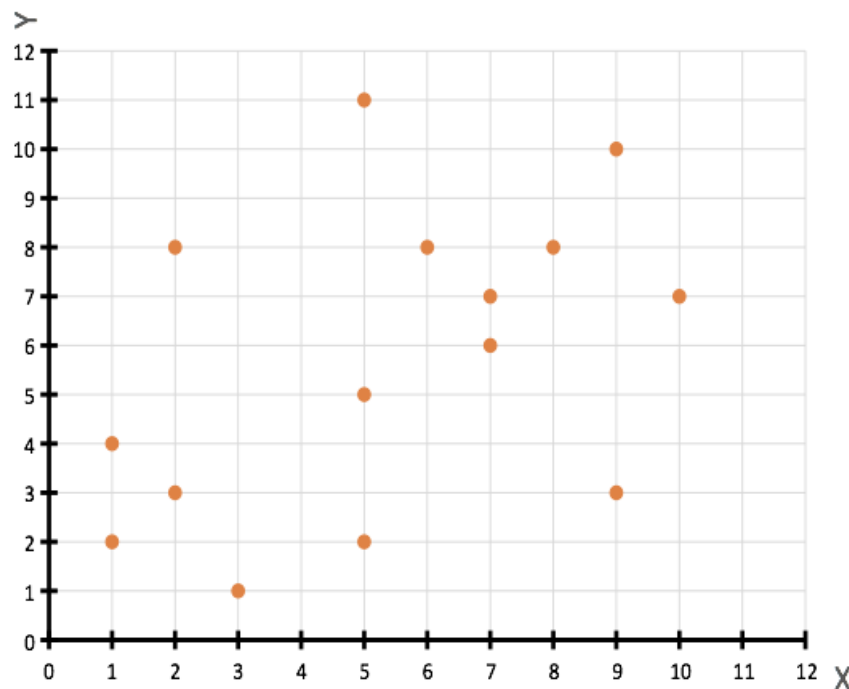
$$P_y = \sum_{i=1}^n p_{iy} / n$$



$$P'_1 = (7.3, 7.2)$$

$$P_x = \sum_{i=1}^n p_{ix} / n$$

$$P'_2 = (1.8, 4.6)$$



K-means 计算案例

- 4、如果计算得出的新中心点与原中心点一样（质心不再移动），那么结束，否则重新进行第二步过程【经过判断，需要重复上述步骤，开始新一轮迭代】

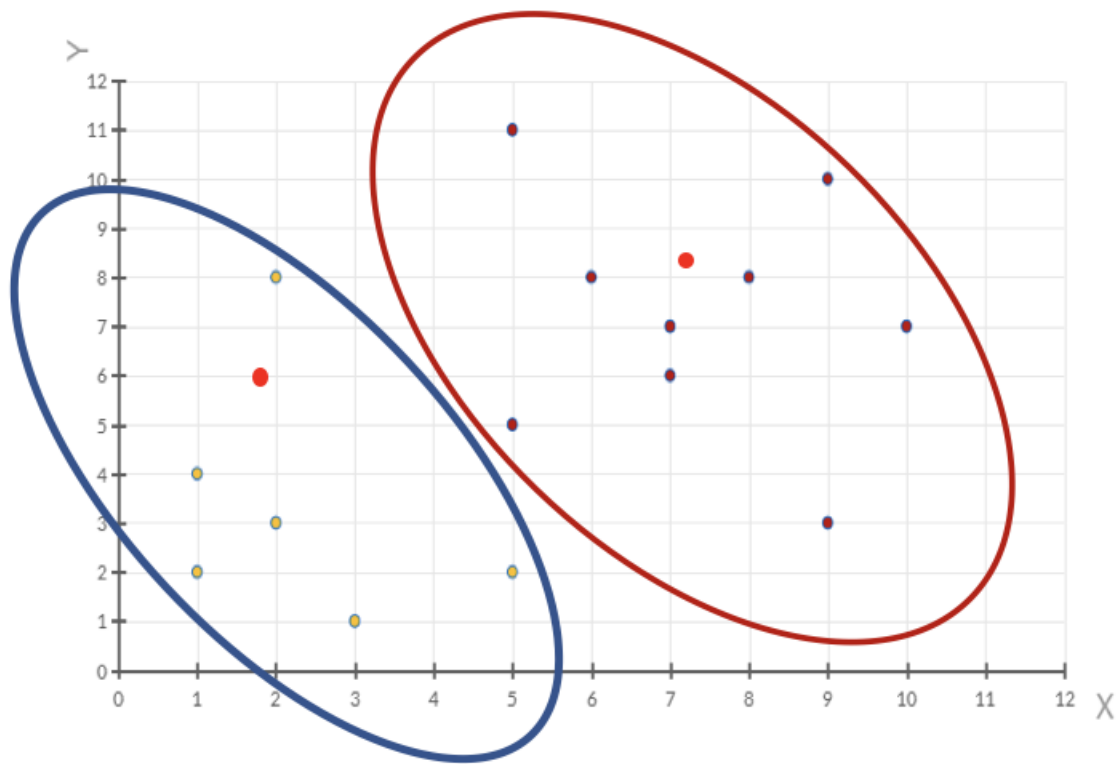
	P'_1 (7.3,7.2)	P'_2 (1.8,4.6)
P1	0.36	5.73
P2	6.75	1.61
P3	1.39	5.40
P4	7.02	1.00
P5	8.16	2.72
P6	7.57	3.79
P7	1.06	7.07

	P'_1 (7.3,7.2)	P'_2 (1.8,4.6)
P8	3.24	9.00
P9	2.82	8.54
P10	3.18	3.22
P11	1.32	5.39
P12	4.66	7.38
P13	5.25	3.41
P14	4.30	7.16
P15	5.25	3.41

P'_1	P1	P3	P7	P8	P9	P10	P11	P12	P14
P'_2	P2	P4	P5	P6	P13	P15			

K-means 计算案例

- 5、当每次迭代结果不变时，认为算法收敛，聚类完成，K-Means一定会停下，不可能陷入一直选质心的过程。



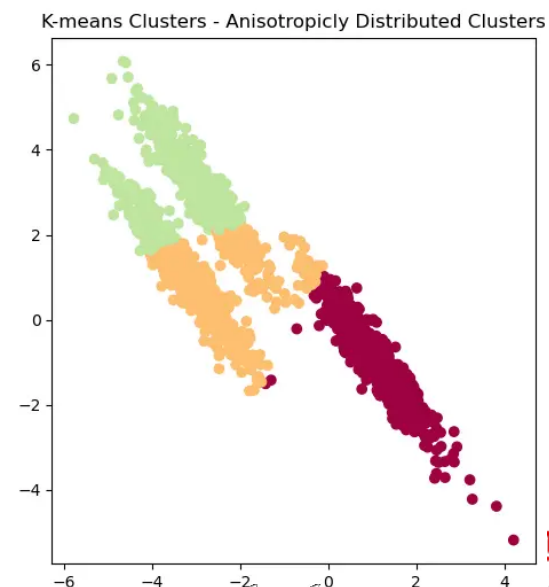
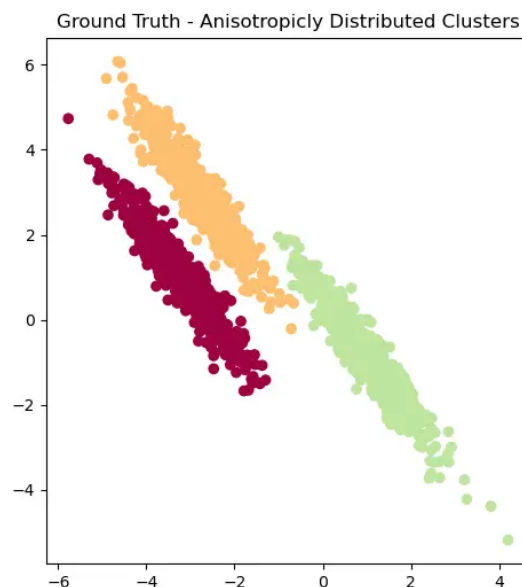
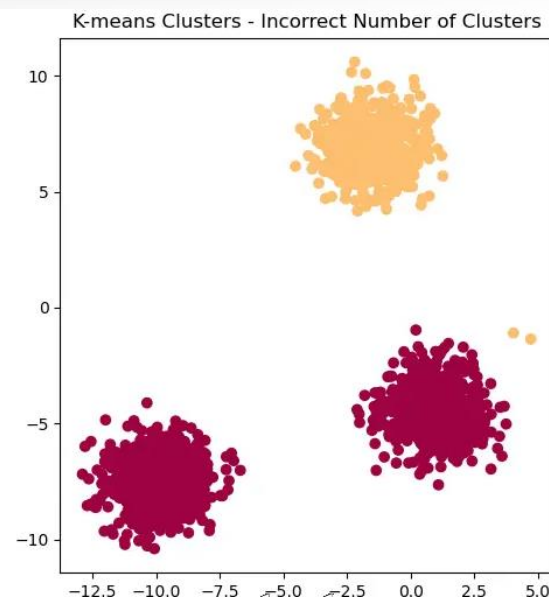
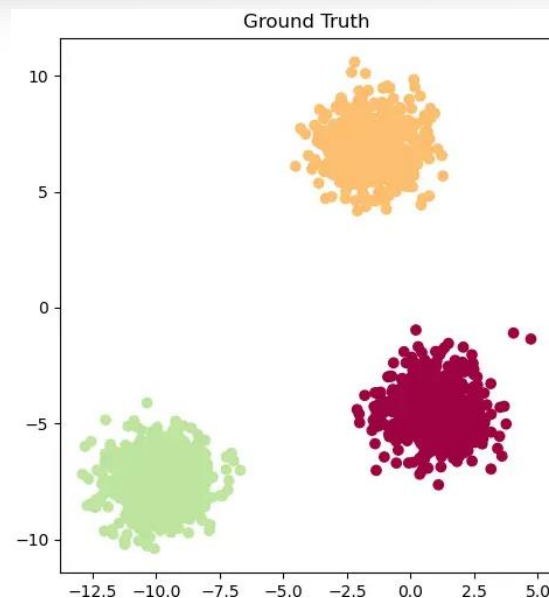
K-means 算法优缺点

■ 优点:

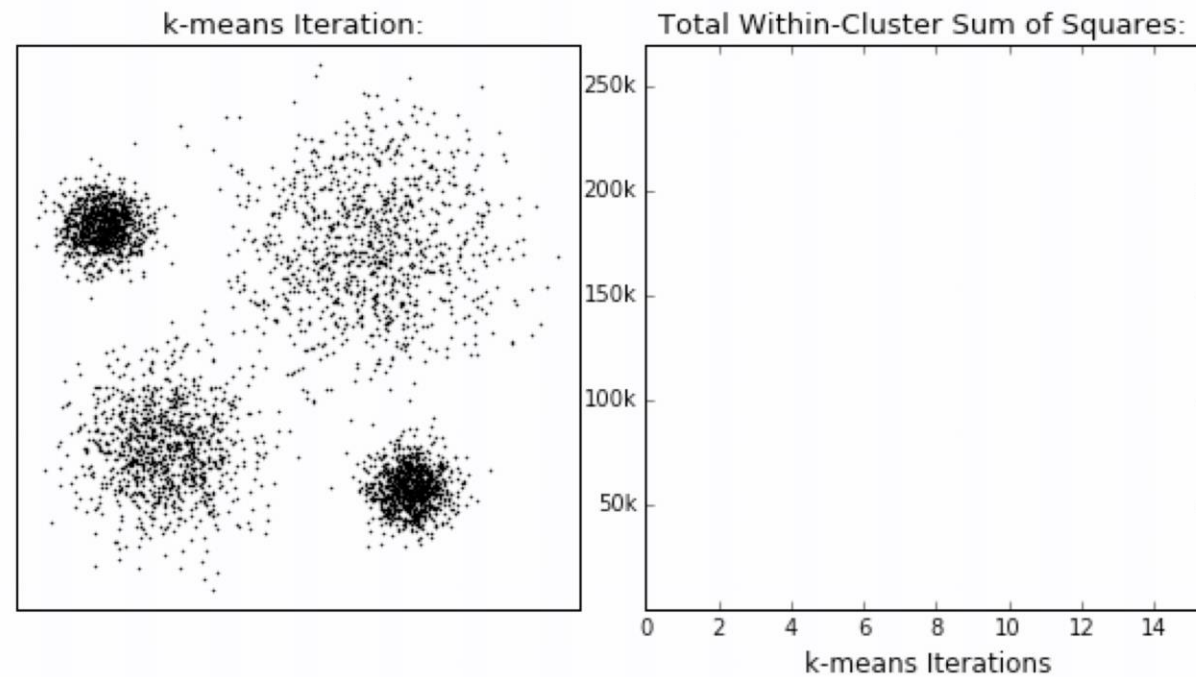
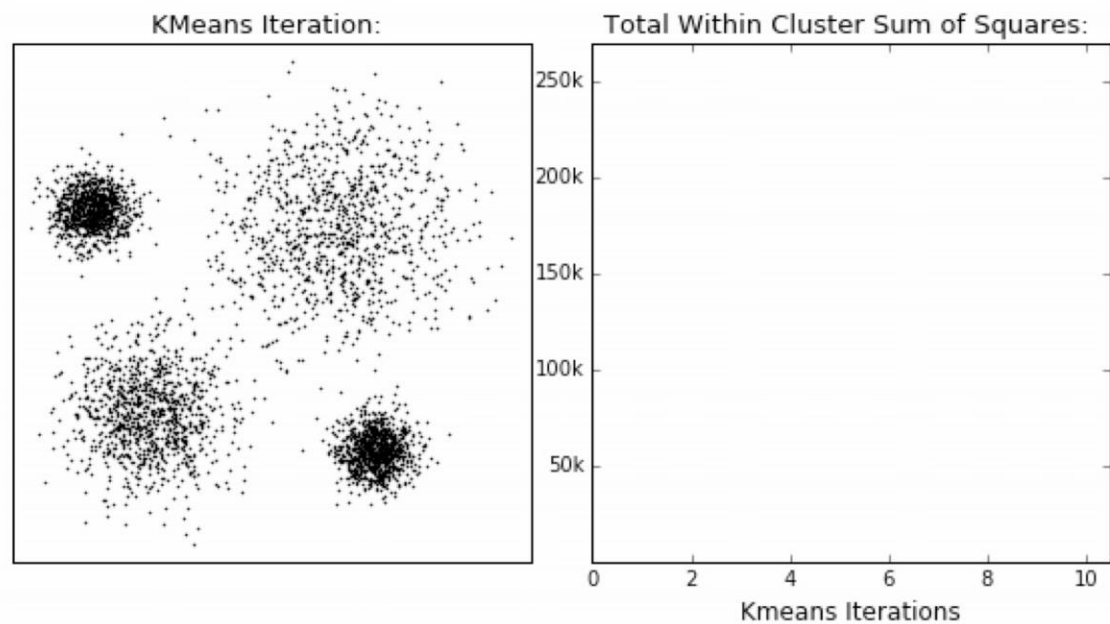
- 原理简单，速度快
- 一般情况下效果不错

■ 缺点:

- 超参数K的选择非常敏感
- 对于Cluster簇的分布形状非常敏感
- 结果不稳定，初始值（种子）的选定对结果有一定的影响



K-means 算法优缺点



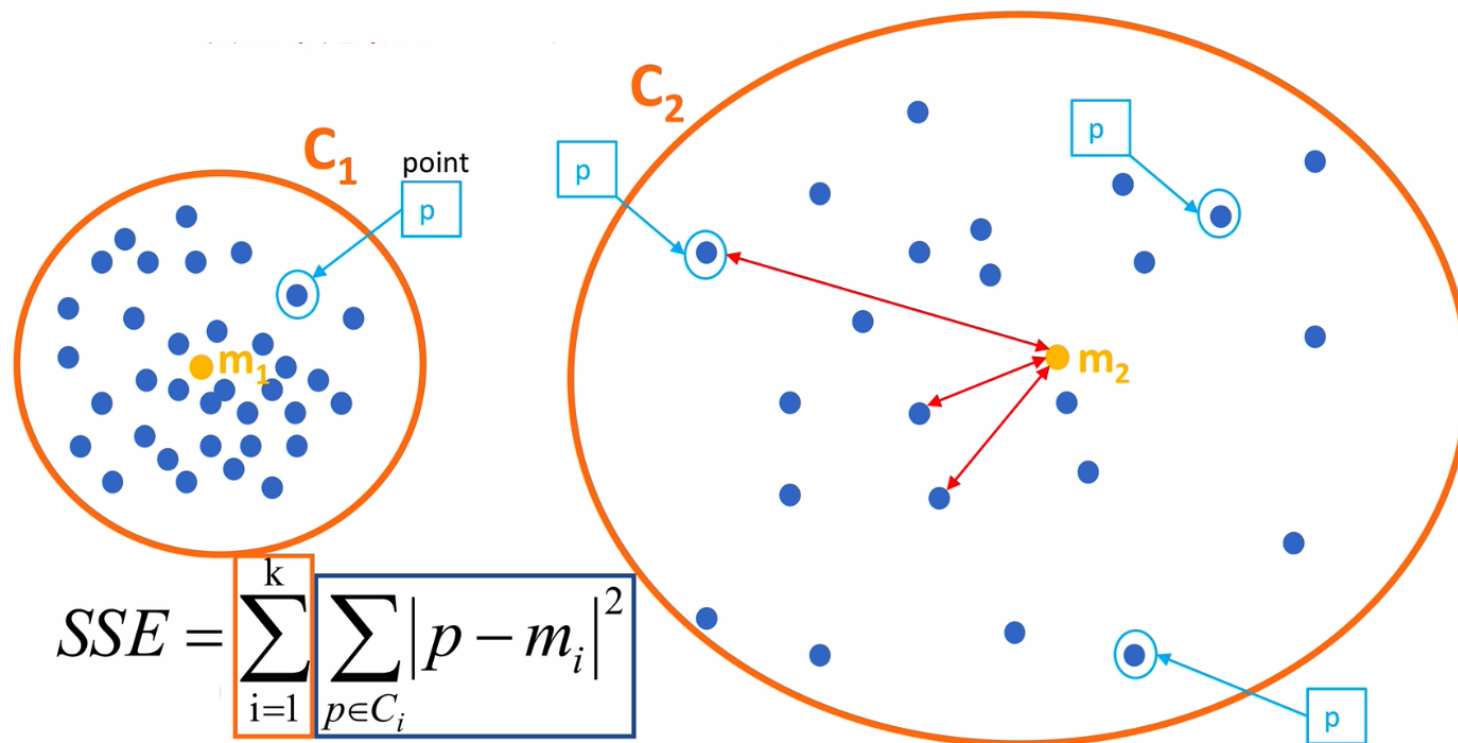
聚类效果评估

- 误差平方和 (SSE)
- 轮廓系数 (Silhouette Coefficient)
- CH系数 (Calinski-Harabasz Index)

聚类效果评估

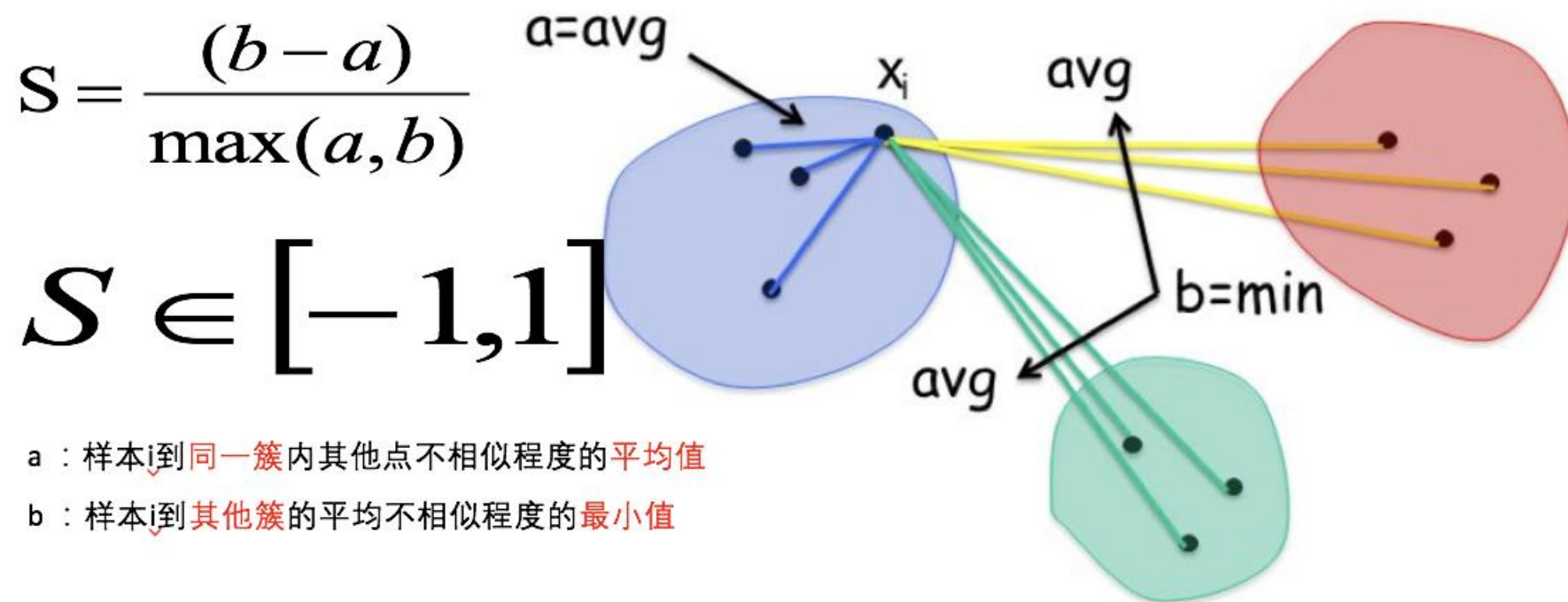
- **误差平方和 (SSE)**
- 越小越好，会随着K的增大而减小

$$SSE = \sum_{i=1}^k \sum_{p \in C_i} |p - m_i|^2$$



聚类效果评估

- **轮廓系数 (Silhouette Coefficient)**
- 结合了聚类的**凝聚度** (Cohesion) 和**分离度** (Separation) , 用于评估聚类的效果
- **平均轮廓系数**的取值范围为 **$[-1,1]$** , 系数越**大**, 聚类效果越**好**。



聚类效果评估

■ CH系数 (Calinski-Harabasz Index)

- 类内部的方差越小越好，类间的方差越大越好（换句话说：类内聚合、类间离散）。
- 用尽量少的类别聚类尽量多的样本，同时获得较好的聚类效果。

■ 越大越好

$$s(k) = \frac{\text{tr}(B_k)}{\text{tr}(W_k)} \frac{m-k}{k-1}$$

类间协方差矩阵

类内协方差矩阵

$$\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{pmatrix}$$

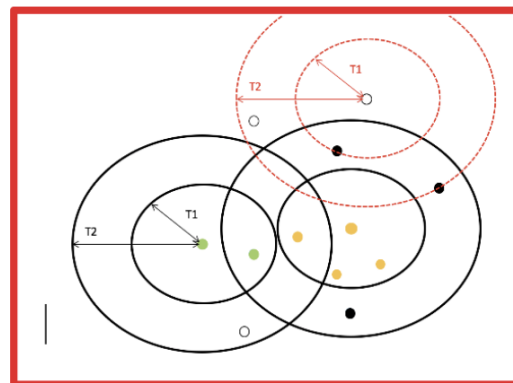
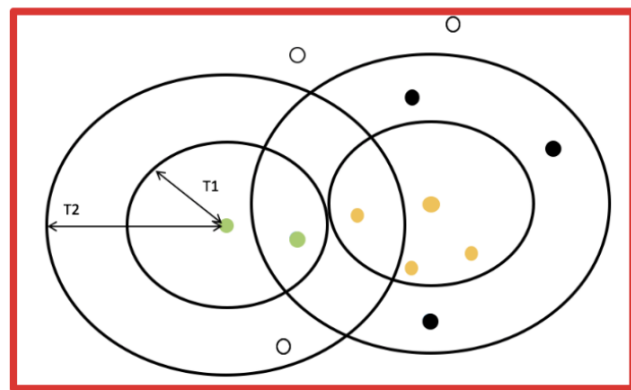
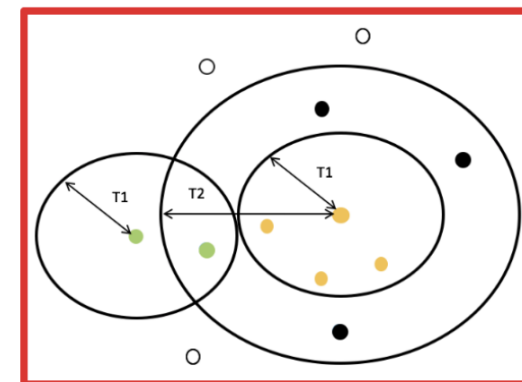
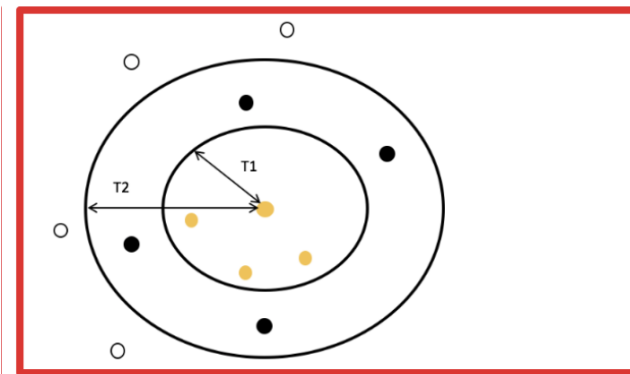
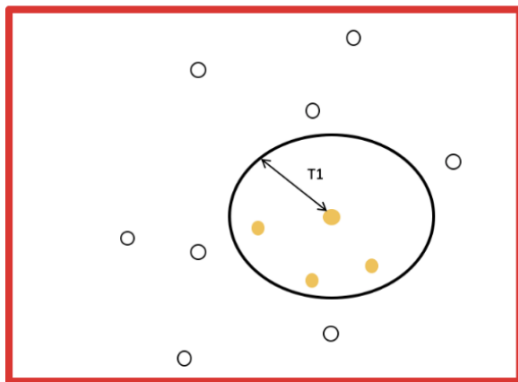
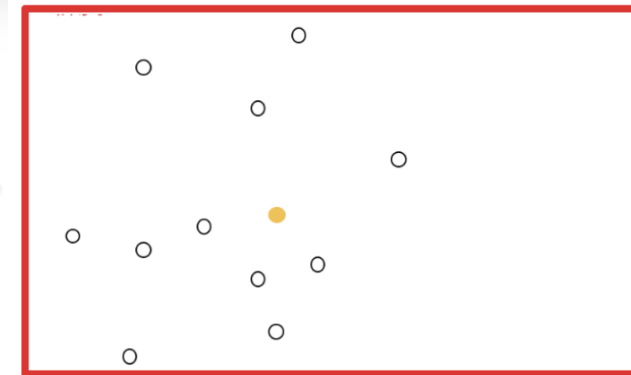
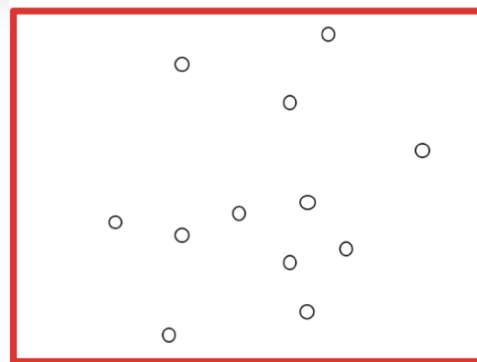
迹，定义为： $a_{11} + a_{22} + \cdots + a_{nn}$

K-means 算法优化

- **canopy** 粗聚类
- **K-means++** 选择初始点
- **二分K-means** 弱化初始中心点影响
- **K-medoids** 中位数解决平均值容易受异常值影响问题
- **肘部法** 帮助选择最优k值

K-means 算法优化: **canopy** 粗聚类

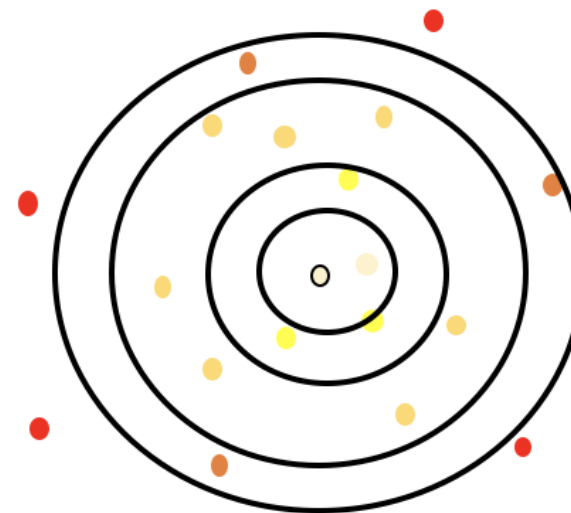
- 需要两个超参数**T1**和**T2**, 要求 **$T1 < T2$**
- 实际工作中搭配: **canopy+kmeans**



K-means 算法优化: K-means++ 选择初始点

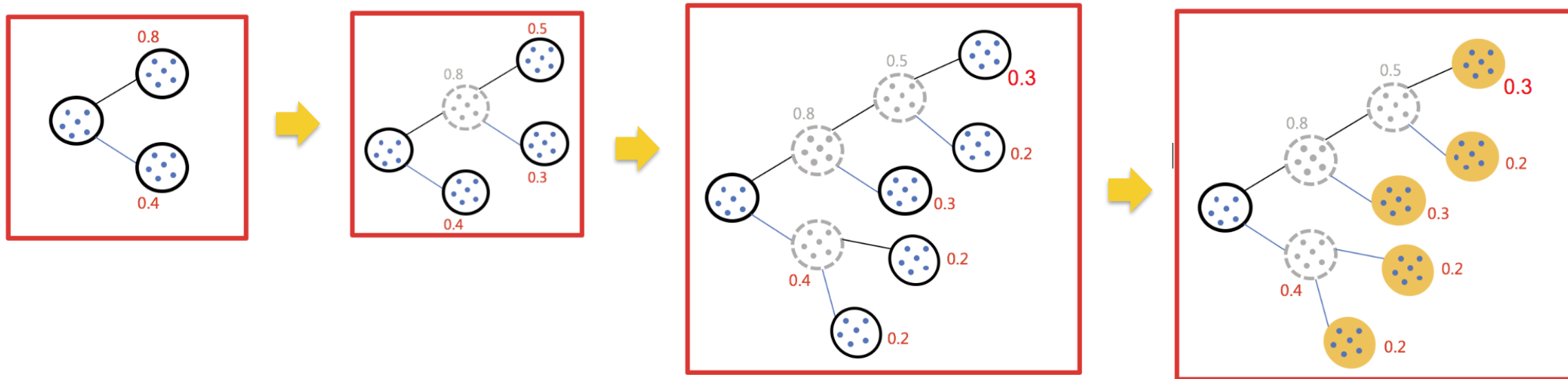
- Step 1: 随机选一个点作为初始中心 C_1
- Step 2: 计算每个点与当前已有中心之间的最短距离, 用 $D(x)$ 表示; 接着计算每个点被选为下一个中心的概率 $P = \frac{D(x)^2}{\sum_{x \in X} D(x)^2}$ 。最后, 轮盘法选出下一个中心点;
- Step 3: 重复第2步直到选择出K个中心;

$$P(A) > P(B) > P(C) > P(D) > P(E)$$



K-means 算法优化：二分K-means 弱化初始中心点影响

- 1.所有点作为一个簇
- 2.将该簇**一分为二**
- 3.选择能**最大限度降低SSE**（误差平方和）的簇划分为两个簇。
- 4.以此进行下去，直到簇的数目**等于**用户给定的数目**k**为止。



K-means 算法优化: K-medoids

- **中位数**解决**平均值**容易受**异常值**影响问题
- **平均值**作为中心点--》**中位数**作为中心点

- 初始化K个**初始中心点**
- E步: 计算每个点到K个中心点的距离, 离谁近就给谁 (打标签)
- M步: 计算每个簇的**均值点**作为新的中心点
- 不断迭代EM步, 直到中心点不在移动

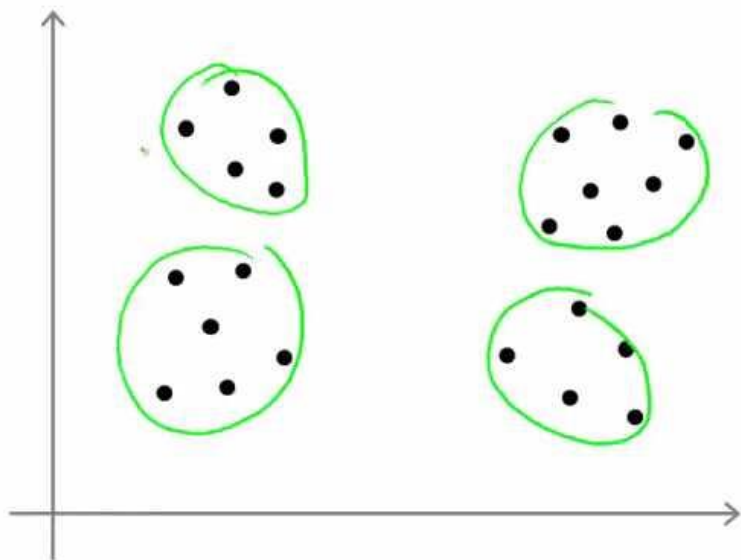
K-means

K-medoids

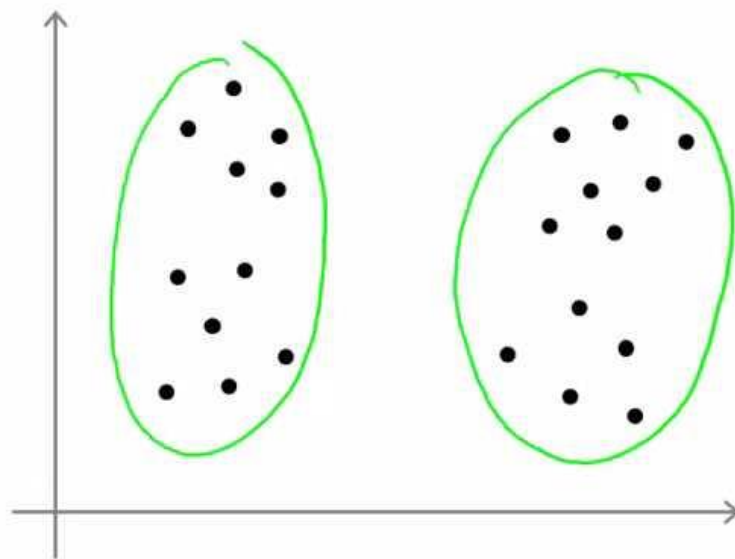
- 初始化K个**初始中心点**
- E步: 计算每个点到K个中心点的距离, 离谁近就给谁 (打标签)
- M步: 计算每个簇的**中位数点**作为新的中心点
- 不断迭代EM步, 直到中心点不在移动

K-means 算法优化: 肘部法 帮助选择最优k值

What is the right value of K?



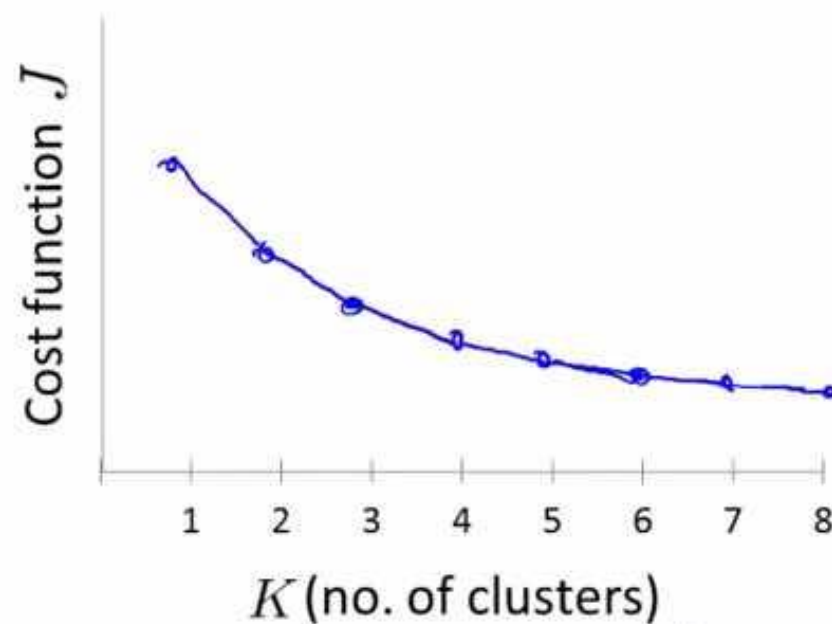
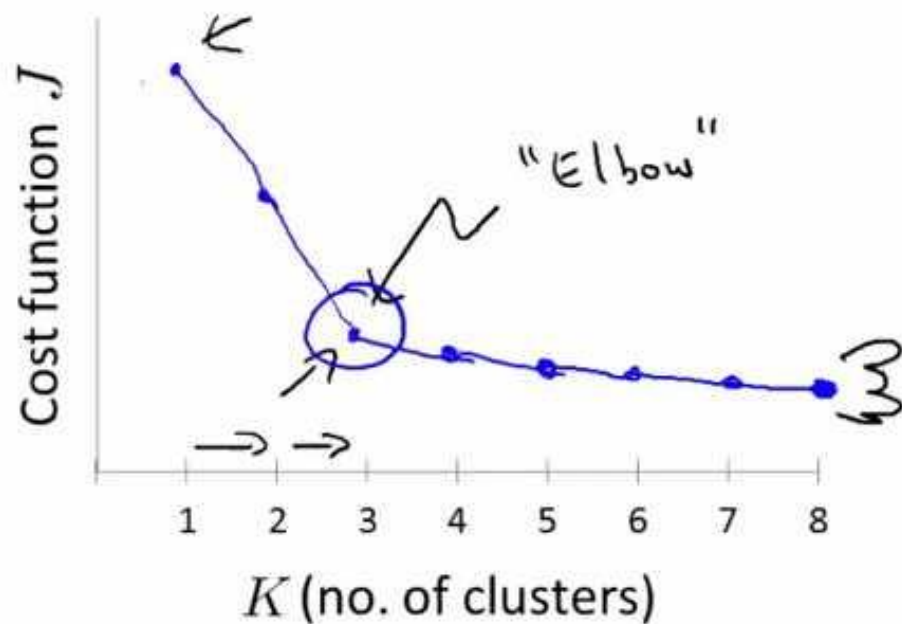
What is the right value of K?



K-means 算法优化: 肘部法 帮助选择最优k值

Choosing the value of K

Elbow method:



导入工具包，加载数据

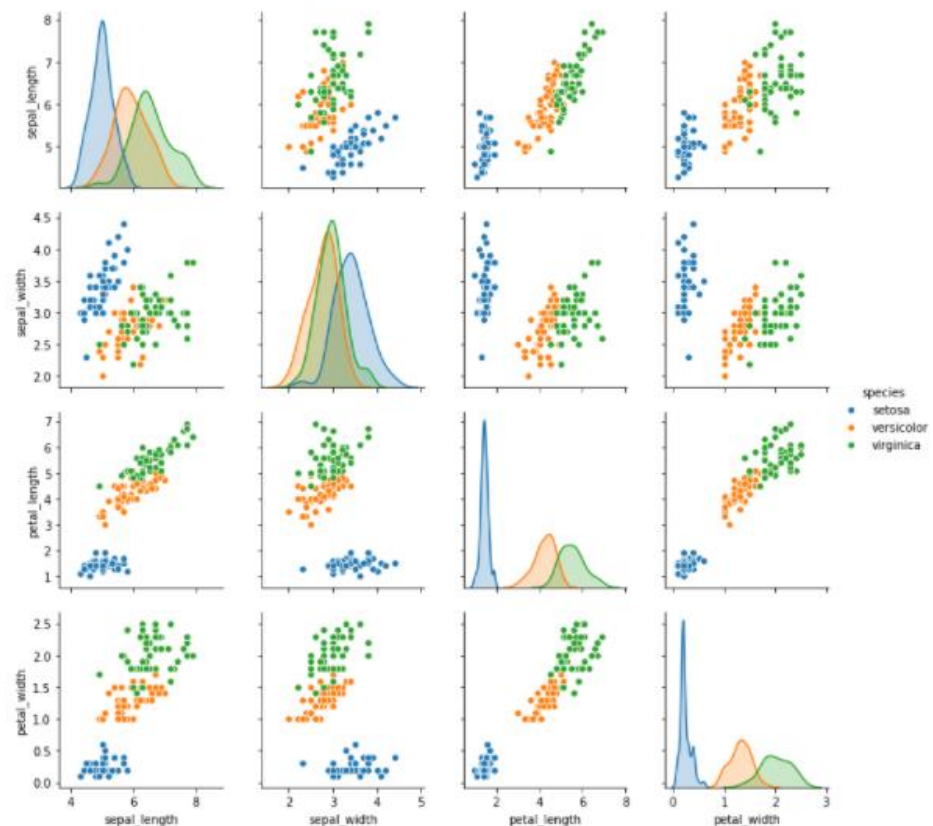
```
# 导入相关包
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.cluster import KMeans
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

```
# 做数据准备，并查看iris数据特点名
# Step1: 数据准备
# 读取iris数据
iris = datasets.load_iris()
X = iris.data
X_names = iris.feature_names
y = iris.target
print(X_names)
```

```
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

查看分布情况

```
# 对数据集中的X变量进行两两作图，查看点的分布情况  
# 使用seaborn的pairplot图  
iris = sns.load_dataset("iris") # 使用了seaborn自带的iris数据集，sklearn的iris把X和y拆开了，不适合直接使用  
sns.pairplot(iris, hue="species")
```



模型训练与预测

[illegible]

```
# Step3: 模型预测
# 使用predict方法对指定的数据集x进行预测
cluster_label = kmeans.predict(X)
cluster_label
```

```
array([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
       [0, 0, 0, 0, 0, 0, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
       [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
       [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 1, 1, 1, 1, 2, 1, 1, 1],
       [1, 1, 1, 2, 2, 1, 1, 1, 1, 2, 1, 2, 1, 2, 1, 1, 2, 2, 1, 1, 1, 1],
       [1, 2, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 2, 1, 1, 1, 1])
```

合并数据集

我们看到针对训练数据集X的预测结果与模型中的labels_结果是一致的。

把labels_合并到数据集X中去

```
labels = kmeans.labels_.reshape(len(kmeans.labels_), 1)
```

```
cluster_result = pd.DataFrame(np.concatenate((X, labels), axis=1), columns=X_names+['cluster'])
```

```
cluster_result.head(10)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	cluster
0	5.1	3.5	1.4	0.2	0.0
1	4.9	3.0	1.4	0.2	0.0
2	4.7	3.2	1.3	0.2	0.0
3	4.6	3.1	1.5	0.2	0.0
4	5.0	3.6	1.4	0.2	0.0
5	5.4	3.9	1.7	0.4	0.0
6	4.8	3.4	1.4	0.3	0.0
7	5.0	3.4	1.5	0.2	0.0
8	4.4	2.9	1.4	0.2	0.0
9	4.9	3.1	1.5	0.1	0.0

使用误分类矩阵对结果予以对比

```

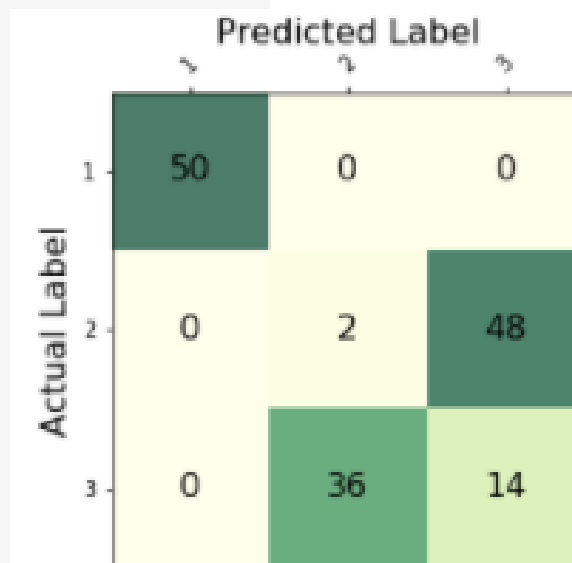
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
%matplotlib inline

# 定义了一个更美观误分类矩阵的函数
def show_confusion_matrix(cnf_matrix, class_labels):
    plt.matshow(cnf_matrix, cmap=plt.cm.YlGn, alpha=0.7)
    ax = plt.gca()
    ax.set_xlabel('Predicted Label', fontsize=16)
    ax.set_xticks(range(0, len(class_labels)))
    ax.set_xticklabels(class_labels, rotation=45)
    ax.set_ylabel('Actual Label', fontsize=16, rotation=90)
    ax.set_yticks(range(0, len(class_labels)))
    ax.set_yticklabels(class_labels)
    ax.xaxis.set_label_position('top')
    ax.xaxis.tick_top()

    for row in range(len(cnf_matrix)):
        for col in range(len(cnf_matrix[row])):
            ax.text(col, row, cnf_matrix[row][col], va='center', ha='center', fontsize=16)

# 误分类矩阵
class_labels = [1, 2, 3]
cnf_matrix = confusion_matrix(y, cluster_label) #cluster_label是聚类结果, y是真实的类标签
show_confusion_matrix(cnf_matrix, class_labels)
plt.show()

```



算法优化

算法优化, 寻找最佳K值

Step2: 肘部法则调参

我们使用scipy.spatial.distance.cdist函数来计算两个点之间的欧式距离, 尝试K的取值从1-10, 相当于针对10个不同的K值构建10个模型, 然后分别评估他们

计算K值从1-10对应的损失函数值 (畸变程度)

from scipy.spatial.distance import cdist #导入计算欧式距离的库

K=range(1, 11) #K的取值从1-10

distortions=[]

针对不同的K值循环建模并计算损失函数

for k in K:

 kmeans=KMeans(n_clusters=k)

 kmeans.fit(X)

 distortions.append(sum(np.min(cdist(X, kmeans.cluster_centers_, 'euclidean'), axis=1)))

作图

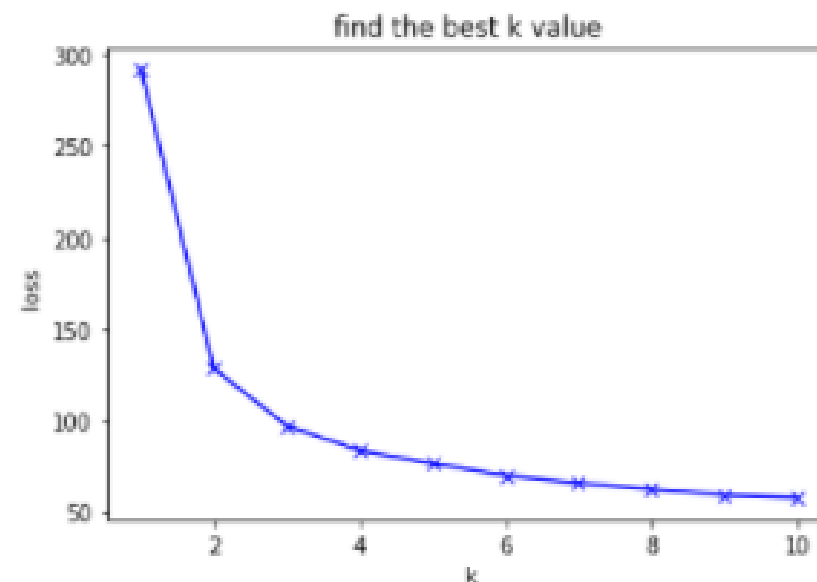
plt.plot(K, distortions, 'bx-')

plt.xlabel('k')

plt.ylabel("loss")

plt.title("find the best k value")

Text(0.5, 1.0, 'find the best k value')



K-means 小结

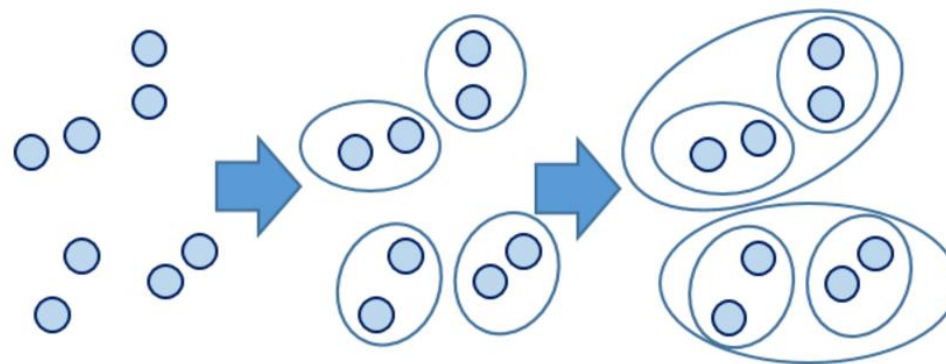
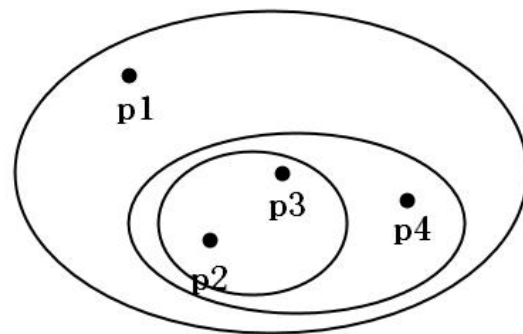
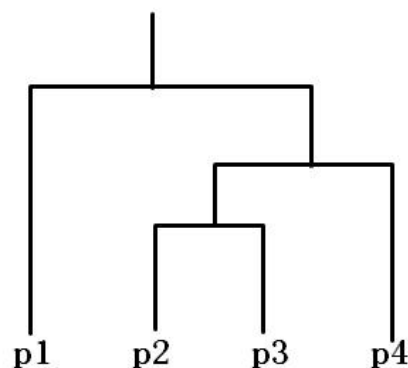
- **原理**: E (中心点求类别) M (类别计算中心点) 迭代
- **优点**: 简单、快速、有效
- **缺点**: k值不好调、初始点影响结果
- **评估**: SSE, SC, CH
- **优化**: canopy, k-means++, 二分kmeans, K-medoids, 肘部法

1
2
chapter

层次聚类

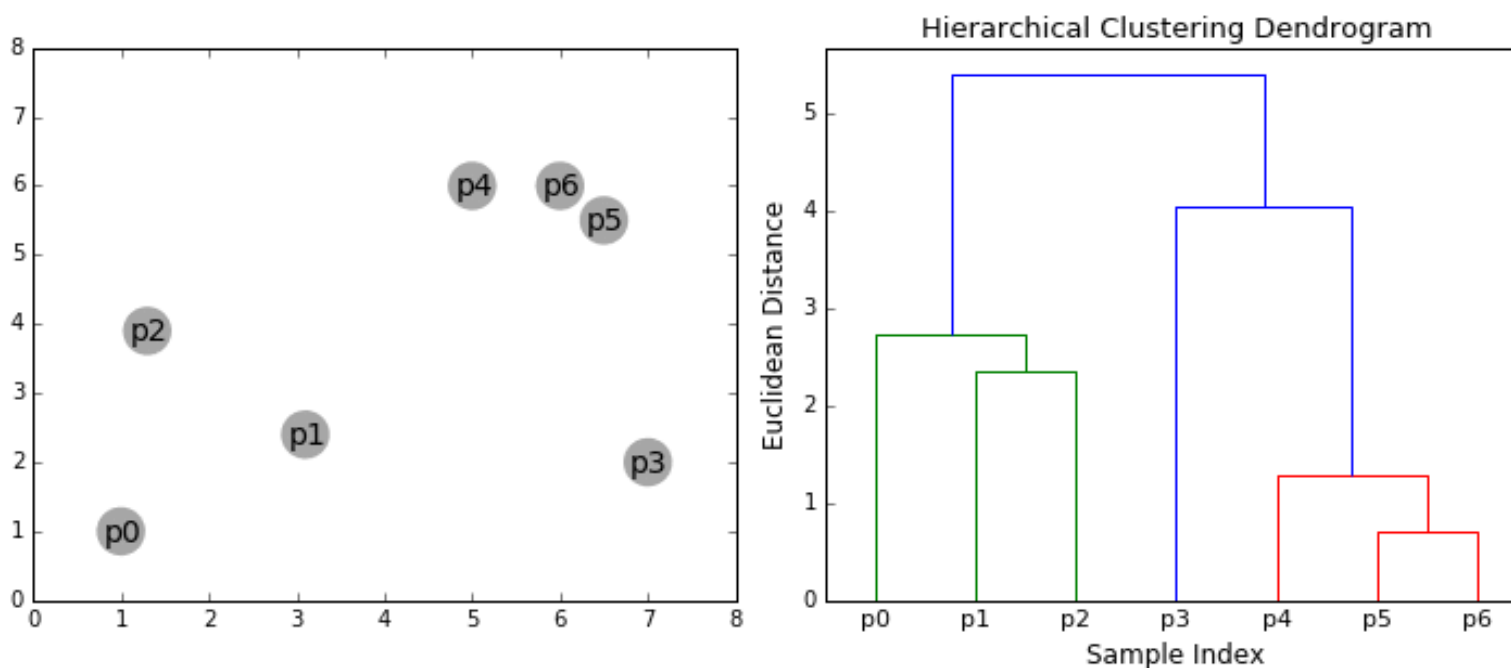
层次聚类

- **层次聚类(Hierarchical Clustering)**是聚类算法的一种，通过计算不同类别数据点间的相似度来创建一棵**有层次的嵌套聚类树**。在聚类树中，不同类别的**原始数据点**是树的**最低层**，树的**顶层**是一个**聚类的根节点**。
- 基于层次的聚类算法 (Hierarchical Clustering) 可以是**凝聚的 (Agglomerative)** 或者**分裂的 (Divisive)**，取决于层次的划分是“**自底向上**”还是“**自顶向下**”。



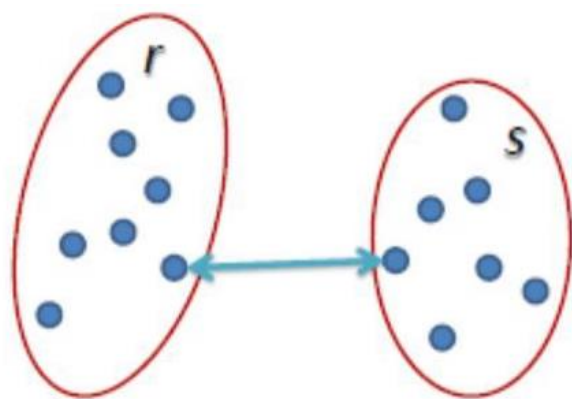
Agglomerative 层次聚类

- 又称**自底向上 (bottom-up)** 的层次聚类，**每一个对象最开始都是一个 cluster**，每次按**一定的准则**将**最相近**的两个 cluster **合并**生成一个新的 cluster，如此往复，直至最终所有的对象都属于一个 cluster。



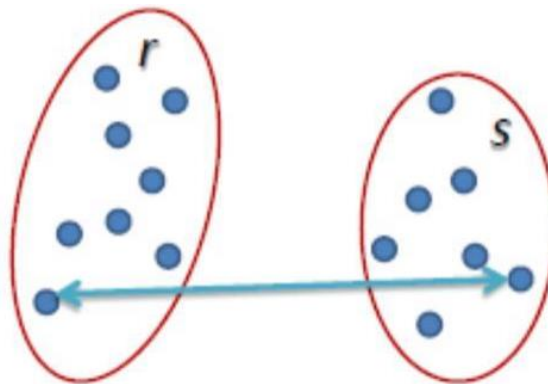
Agglomerative 中的簇间距离计算 Take the two closest clusters and make them one cluster

- **Single Linkage**: 两个簇中两点之间的**最短**距离
- **Complete Linkage**: 两个簇中两点之间的**最长**距离
- **Average Linkage**: 两个簇中点之间两两距离的**平均值**



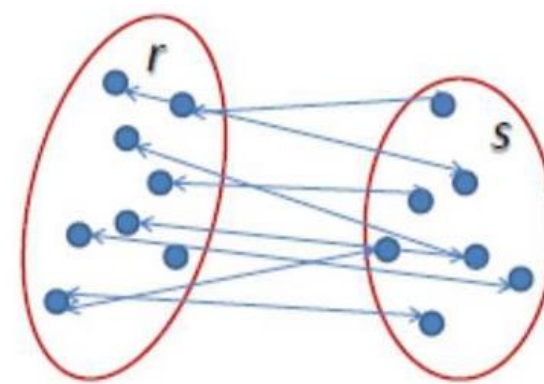
$$L(r, s) = \min(D(x_{ri}, x_{sj}))$$

Single Linkage



$$L(r, s) = \max(D(x_{ri}, x_{sj}))$$

Complete Linkage

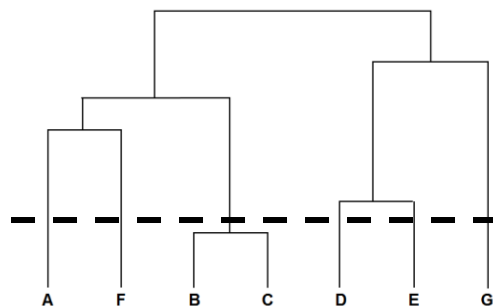


$$L(r, s) = \frac{1}{n_r n_s} \sum_{i=1}^{n_r} \sum_{j=1}^{n_s} D(x_{ri}, x_{sj})$$

Average Linkage

Agglomerative 算法案例

A	16.9
B	38.5
C	39.5
D	80.8
E	82
F	34.6
G	116.1

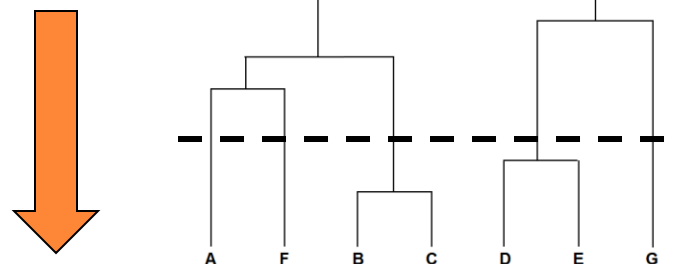


$$D = \sqrt{(B - C)^2} = \sqrt{(38.5 - 39.5)^2}$$

	A	B	C	D	E	F	G
A	0	21.60	22.60	63.90	65.10	17.70	99.20
B	21.60	0	1.00	42.30	43.50	3.90	77.60
C	22.60	1.00	0	41.30	42.50	4.90	76.60
D	63.90	42.30	41.30	0	1.20	46.20	35.30
E	65.10	43.50	42.50	1.20	0	47.40	34.10
F	17.70	3.90	4.90	46.20	47.40	0	81.50
G	99.20	77.60	76.60	35.30	34.10	81.50	0

$$D = \frac{\sqrt{(B - A)^2} + \sqrt{(C - A)^2}}{2} = \frac{21.6 + 22.6}{2}$$

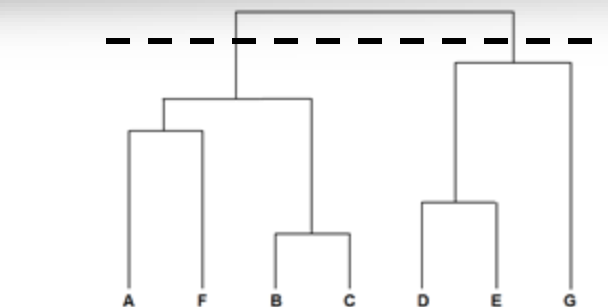
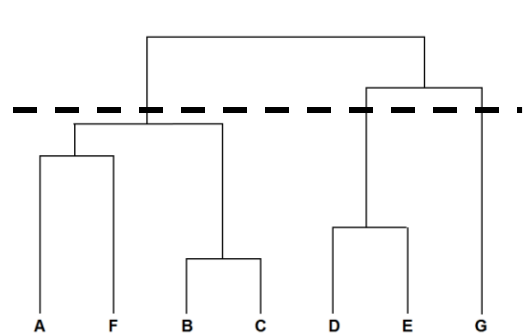
	A	(B,C)	D	E	F	G
A	0	22.10	63.90	65.10	17.70	99.20
(B,C)	22.10	0	41.80	43.00	4.40	77.10
D	63.90	41.80	0	1.20	46.20	35.30
E	65.10	43.00	1.20	0	47.40	34.10
F	17.70	4.40	46.20	47.40	0	81.50
G	99.20	77.10	35.30	34.10	81.50	0



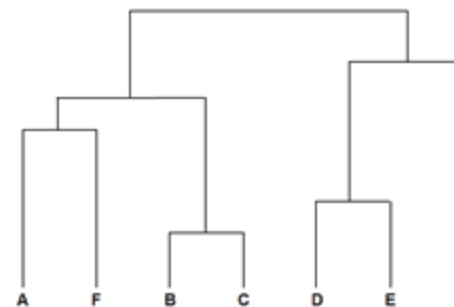
	A	(B,C)	(D,E)	F	G
A	0	22.10	64.50	17.70	99.20
(B,C)	22.10	0	42.40	4.40	77.10
(D,E)	64.50	42.40	0	46.80	34.70
F	17.70	4.40	46.80	0	81.50
G	99.20	77.10	34.70	81.50	0

Agglomerative 算法案例

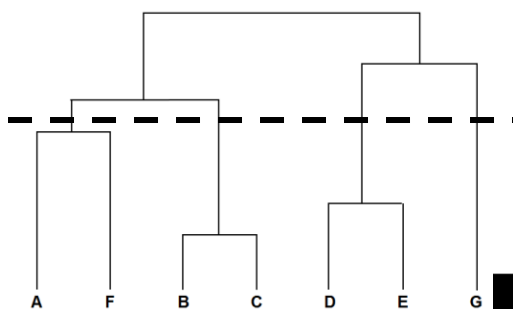
	(A,F,B,C)	(D,E)	G
(A,F,B,C)	0	49.03	83.73
(D,E)	49.03	0	34.70
G	83.73	34.70	0



	(A,F,B,C)	(D,E,G)
(A,F,B,C)	0	60.59
(D,E,G)	60.59	0



	(A,F)	(B,C)	(D,E)	G
(A,F)	0	13.25	55.65	90.35
(B,C)	13.25	0	42.40	77.10
(D,E)	55.65	42.40	0	34.70
G	90.35	77.10	34.70	0

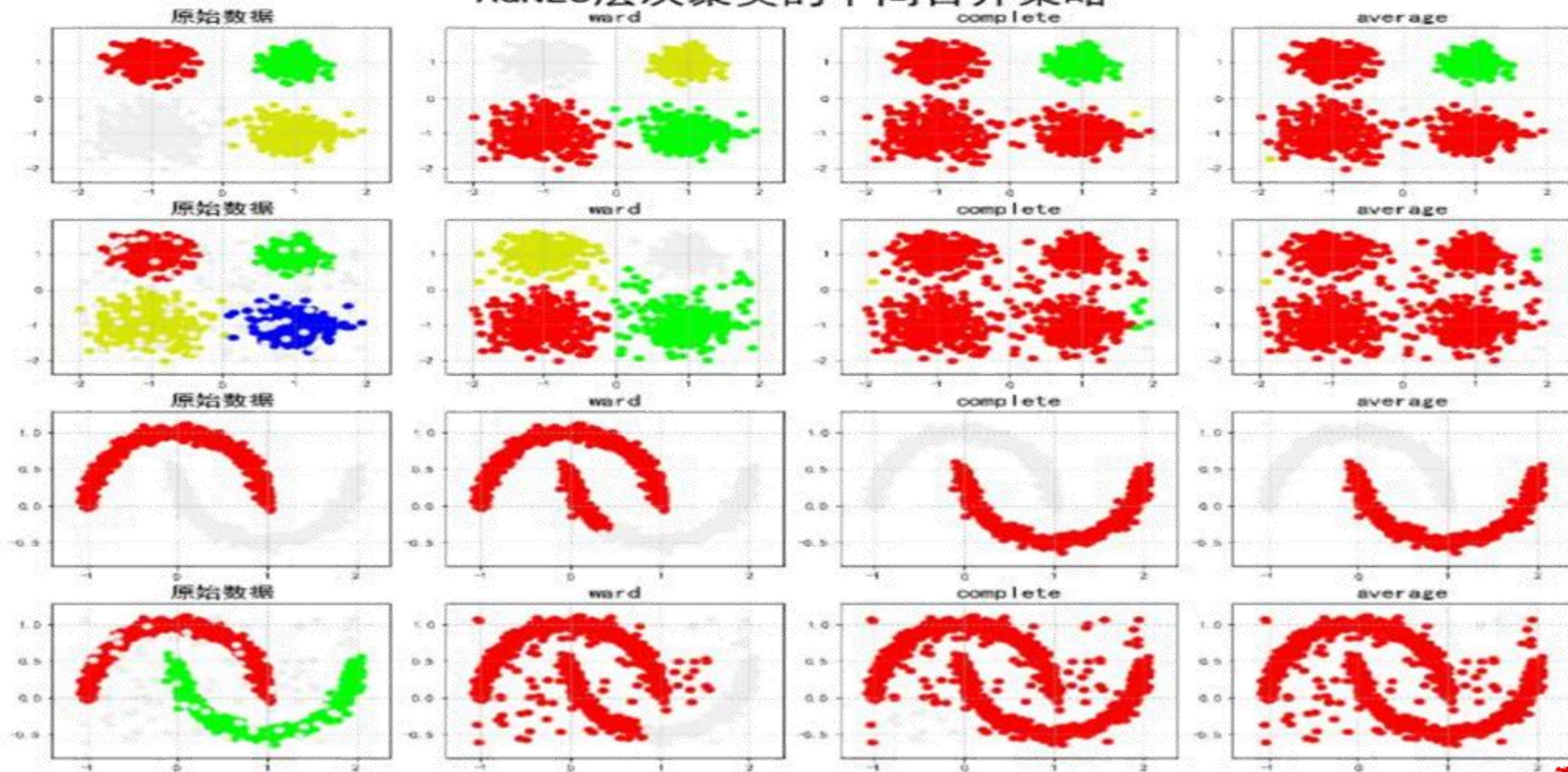


	A	(B,C)	(D,E)	F	G
A	0	22.10	64.50	17.70	99.20
(B,C)	22.10	0	42.40	4.40	77.10
(D,E)	64.50	42.40	0	46.80	34.70
F	17.70	4.40	46.80	0	81.50
G	99.20	77.10	34.70	81.50	0

$$D = \frac{\sqrt{(A-B)^2} + \sqrt{(A-C)^2} + \sqrt{(F-B)^2} + \sqrt{(F-C)^2}}{4}$$

Agglomerative 不同距离聚类效果

AGNES层次聚类的不同合并策略



Divisive 层次聚类

- 又称**自顶向下 (top-down)** 的层次聚类，最开始**所有**的对象均属于**一个 cluster**，每次按**一定的准则**将某个 cluster **划分为多个** cluster，如此往复，直至**每个**对象**都是一个** cluster。

小结

- **两种方式：**凝聚的（Agglomerative）、分裂的（Divisive），
- **三种簇间距离：**Single Linkage最短、Complete Linkage最长、Average Linkage平均
- **优点：**
 - 1、距离和规则的相似度容易定义，限制少
 - 2、不需要预先制定聚类数
 - 3、可以发现类的层次关系
- **缺点：**计算复杂度太高；

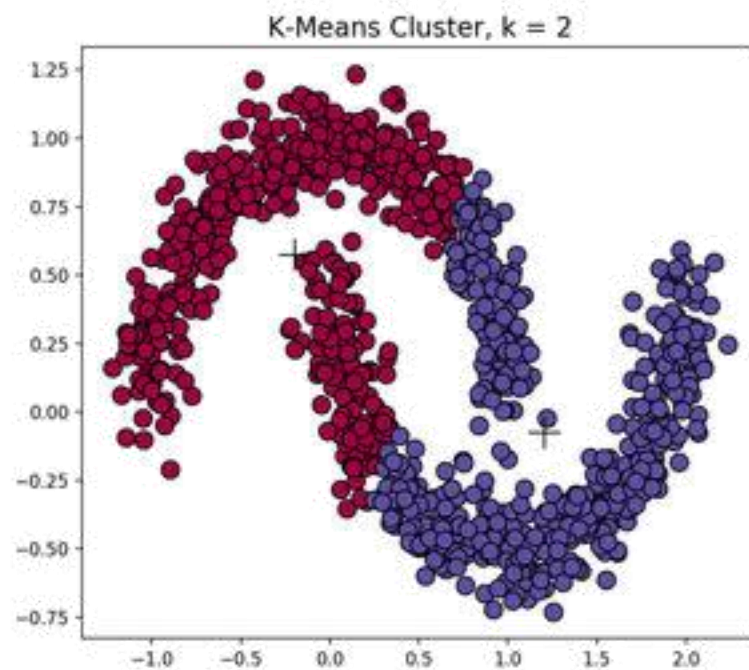
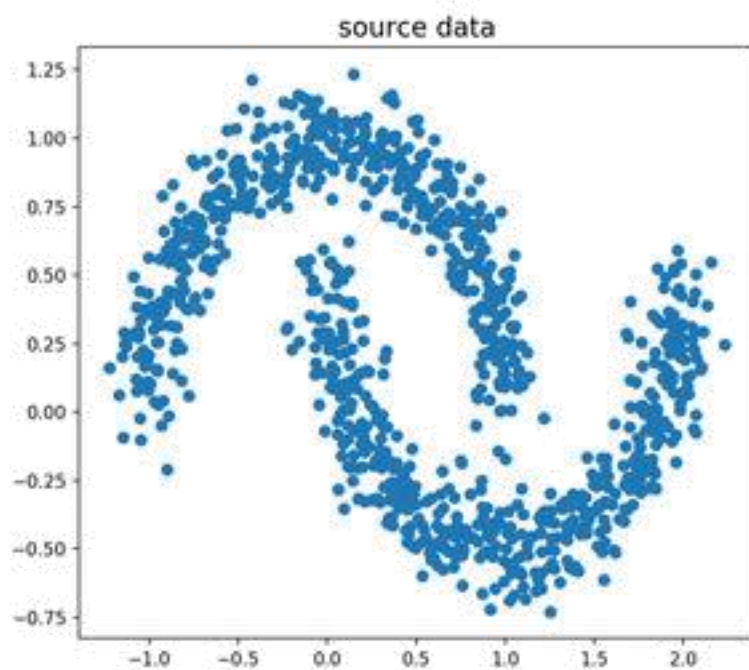
The background features a complex network diagram with numerous nodes (represented by small grey and black dots) connected by thin, light grey lines. These connections form a web-like structure that fills the entire slide, with some areas appearing denser than others. The overall color palette is light grey and white, with the chapter title providing a dark blue contrast.

1
3
chapter

Dbscan 密度聚类

K-means 存在的问题

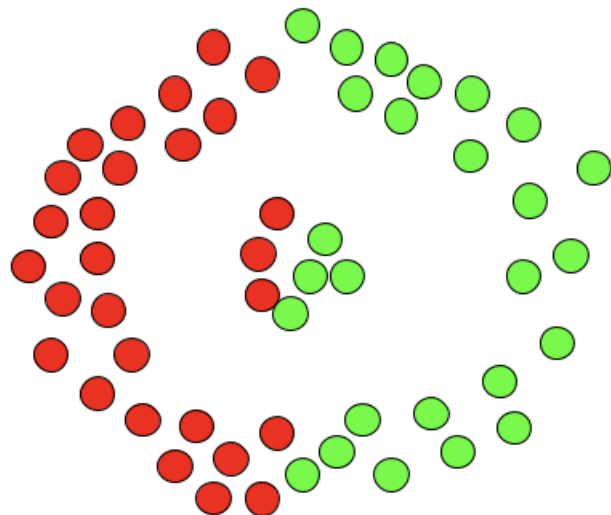
- **k-means算法**对于**凸性数据**具有良**好**的效果，能够根据距离来将数据分为**球状类**的簇，但对于**非凸**形状的数据点，就**无能为力**了，当k-means算法在**环形**数据的聚类时，我们看看会发生什么情况。



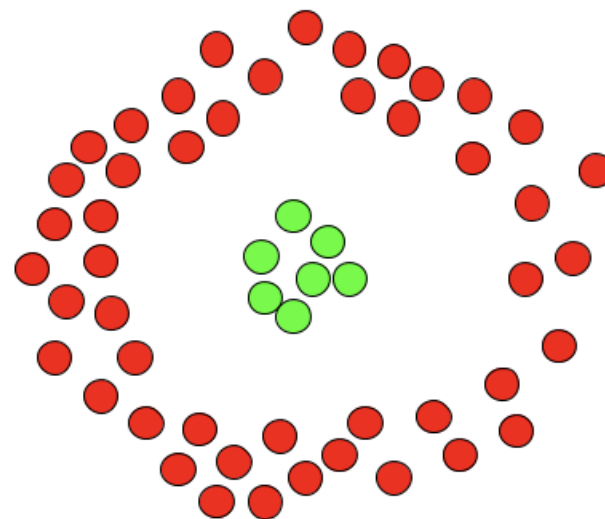
DBSCAN

- **DBSCAN** (Density-Based Spatial Clustering of Applications with Noise, 具有噪声的基于密度的聚类方法) 是一种**基于密度**的空间聚类算法。该算法将**具有足够密度的区域**划分为簇, 并在具有噪声的空间数据库中发现**任意形状**的簇, 它将簇定义为**密度相连**的点的最大集合。

K-Means

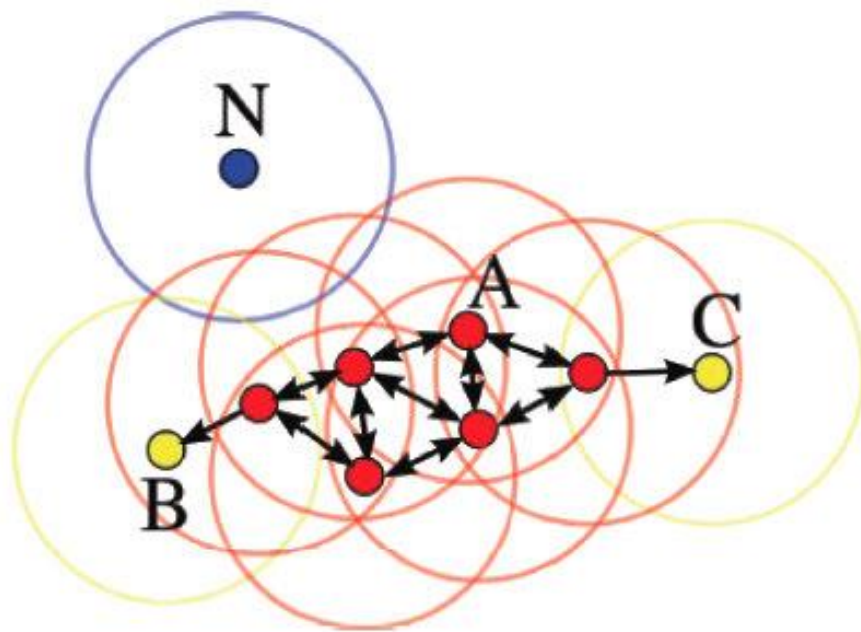


DBSCAN



DBSCAN 核心概念

- **两个参数：** ϵ 半径、minPts 阈值
- **三种点：** 核心点、边界点、离群点
- **密度概念：** 密度、直接密度可达、密度可达、密度相连



DBSCAN 核心概念

集合 $X = \{x^{(1)}, x^{(2)}, \dots, x^{(n)}\}$, ϵ 表示定义密度的邻域**半径**, 设聚类的邻域**密度阈值**为 ***MinPts***

■ **ϵ 邻域(ϵ -neighborhood)** : 给定对象在半径 ϵ 内的区域

$$N_{\epsilon}(x) = \{y \in X : dist(x, y) \leq \epsilon\}$$

■ **密度(density)**: ϵ 邻域中 x 的密度, 是 ϵ 邻域内**点的个数**, 依赖于半径 ϵ

$$p(x) = |N_{\epsilon}(x)|$$

DBSCAN核心概念

■ 核心点(core point):

设 $x \in X$, 若 $\rho(x) \geq M$, 则称 x 为 X 的核心点, 记 X 中所有核心点构成的集合为 X_c , 记所有非核心点构成的集合为 X_{nc} 。

■ 边界点(border point):

若 $x \in X_{nc}$, 且 $\exists y \in X$, 满足

$$y \in N_\epsilon(x) \cap X_c$$

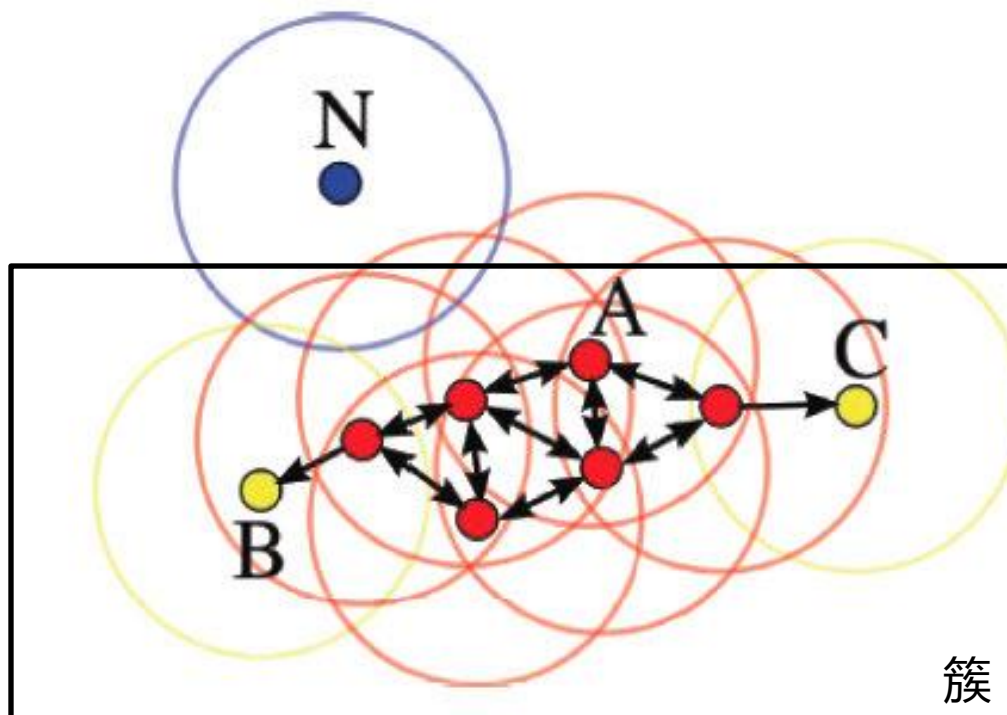
即 x 的 ϵ 邻域中存在核心点, 则称 x 为 X 的边界点, 记 X 中所有的边界点构成的集合为 X_{bd} 。

■ 噪音点(noise point):

集合中除了边界点和核心点之外的点都是噪音点

$$x \in X, x \notin X_c \text{ 且 } x \notin X_{bd}$$

DBSCAN 核心概念



蓝色 (N) : 异常点

红色 (A) : 核心点

黄色 (B/C) : 边界点

黑色方框: 簇

假设我们指定密度阈值 $\text{MinPts} = 4$, 那么图中的红色点就是核心点, 因为它们的 ϵ -邻域里包含最少 4 个点 (包括自己), 由于它们之间相互可达, 它们形成了一个聚类。

图中黄色的点 (B/C) 不是核心点, 但它们在核心点的圆圈内, 所以也属于同一个聚类, 也就是边界点。

而蓝色点既不是核心点, 又不是边界点, 即被称之为异常点。

DBSCAN 核心概念

直接密度可达：设 $x, y \in X$ ，若满足 $y \in N_\epsilon(q)$ 且 $|N_\epsilon(q)| \geq MinPts$ ，则称 y 从 x 直接密度可达。

密度可达：假设存在一串点 $p_1, p_2, \dots, p_n, p_1 = q, p_n = p$ ，使得 p_{i+1} 从 p_i 是直接密度可达的，那么就认为 p 从 q 密度可达。

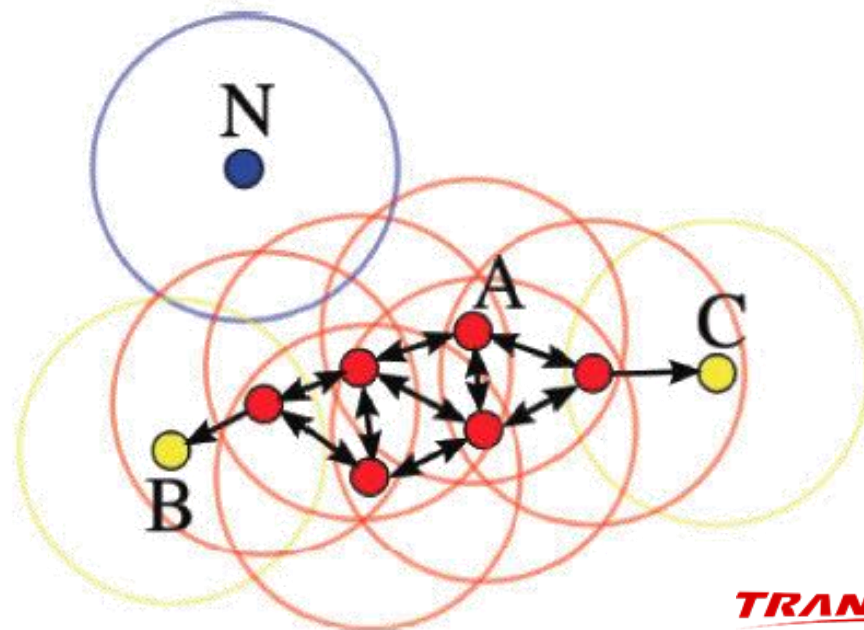
密度相连：假设存在点 o, p, q ，其中 p, q 均从 o 密度可达，那么 p 和 q 密度相连。密度相连具有对称性。

类簇：设非空集合 $C \subset X$ ，若满足： $\forall p, q$ ，

- (1) $p \in C$ ，且 q 从 p 密度可达，那么 $q \in C$ 。

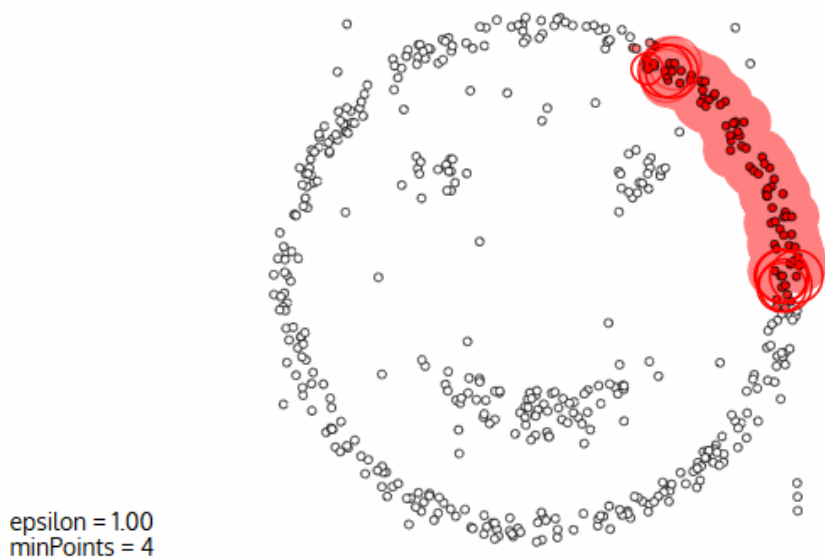
- (2) p 和 q 密度相连。

则称 C 构成一个类簇。



DBSCAN 算法流程

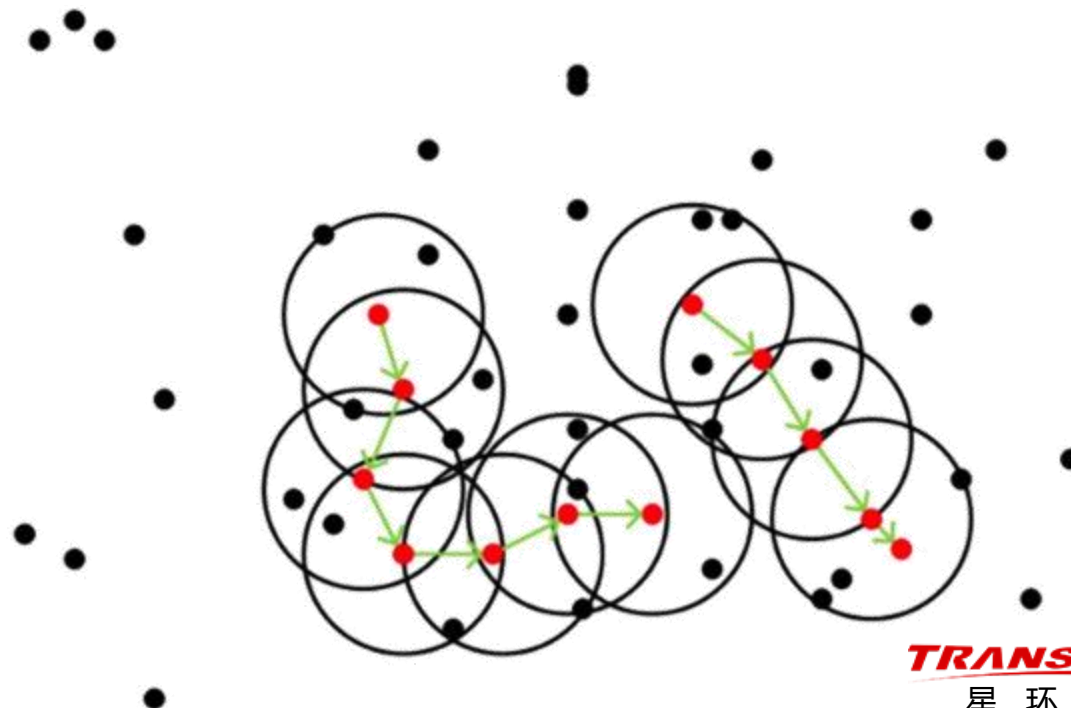
- 如果一个点 x 的 ϵ 邻域包含多于 m 个对象，则创建一个 x 作为核心对象的新簇；
- 寻找并合并核心对象直接密度可达的对象；
- 没有新点可以更新簇的时候，算法结束。



Restart

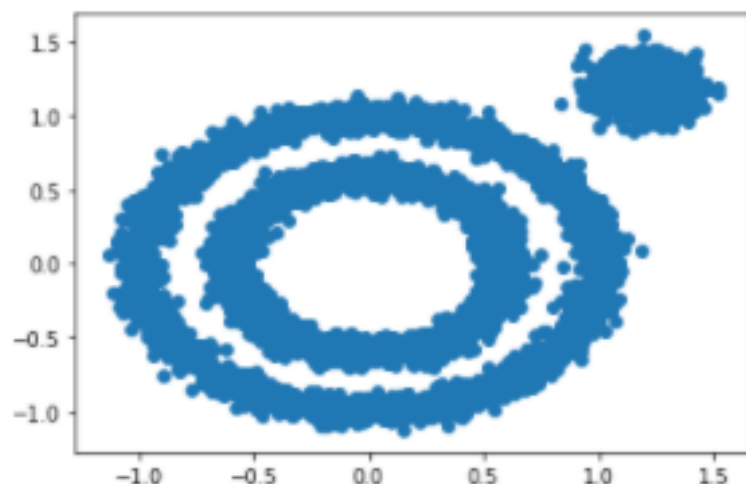


Pause



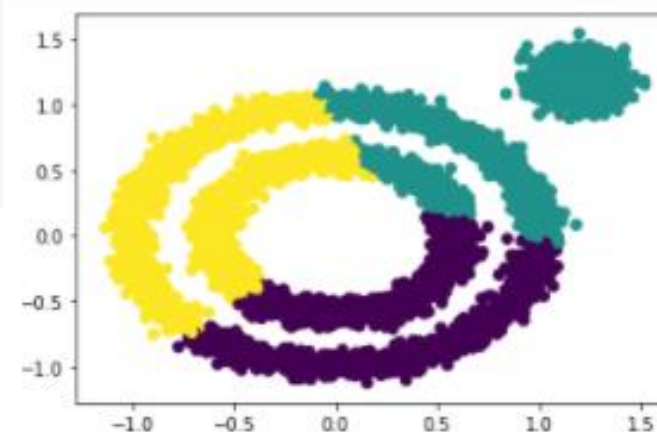
导入工具包，准备数据

```
# 为了更明显的体现DBSCAN与K均值的差异，我们构造一个非凸的数据集，它适合用DBSCAN来聚类而不是适合K均值。  
import numpy as np  
from sklearn import datasets  
import matplotlib.pyplot as plt  
%matplotlib inline  
  
# 构造数据集X，并可视化查看数据结构  
X1, y1 = datasets.make_circles(n_samples=5000, factor=.6, noise=.05)  
X2, y2 = datasets.make_blobs(n_samples=1000, n_features=2, centers=[[1.2, 1.2]], cluster_std=[[.1]], random_state=9)  
X = np.concatenate((X1, X2))  
# 用X1和X2两个变量作散点图  
plt.scatter(X[:, 0], X[:, 1], marker='o')  
plt.show()
```

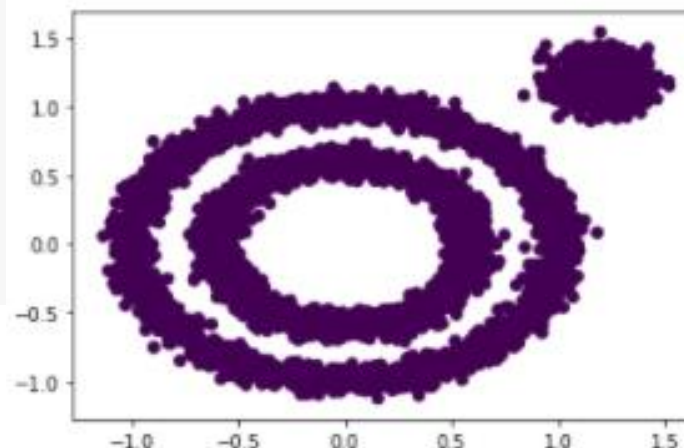


模型的训练

```
# 1. K均值聚类
from sklearn.cluster import KMeans
cluster = KMeans(n_clusters=3).fit_predict(X) # 聚类结果保存为cluster
plt.scatter(X[:, 0], X[:, 1], c=cluster) # 用cluster作为颜色图例显示聚类结果
plt.show()
```

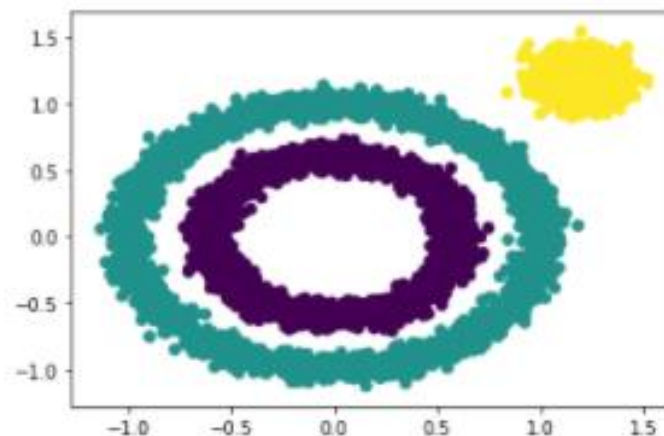


```
# DBSCAN聚类模型
# 一开始我们使用默认参数先看看DBSCAN模型的效果
from sklearn.cluster import DBSCAN
cluster = DBSCAN().fit_predict(X)
plt.scatter(X[:, 0], X[:, 1], c=cluster)
plt.show()
```

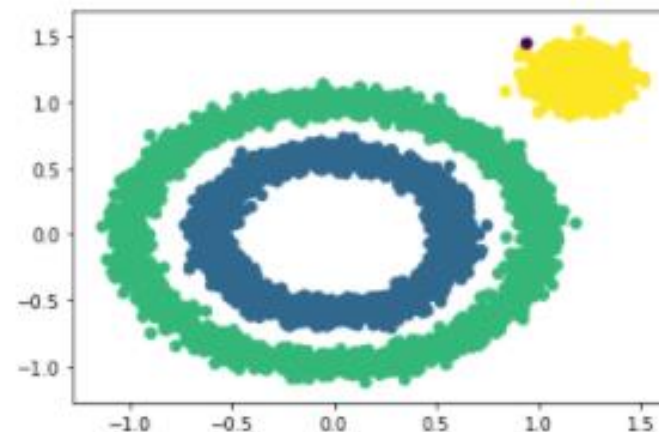


模型的对比

```
# eps默认是0.5, 我们减到0.1看看效果。  
cluster = DBSCAN(eps = 0.1).fit_predict(X)  
plt.scatter(X[:, 0], X[:, 1], c=cluster)  
plt.show()
```



```
#我们现在将min_samples从默认的5增加到10。看看效果  
cluster = DBSCAN(eps = 0.1, min_samples = 10).fit_predict(X)  
plt.scatter(X[:, 0], X[:, 1], c=cluster)  
plt.show()
```



DBSCAN 优缺点

■ 优点:

- 不需要像 Kmeans 那样预先确定K值
- 对异常值**不**敏感
- 可以聚类非线性关系(聚类为任意形状)

■ 缺点:

- 很难在不同密度（有密有疏）的数据中识别集群
- 难以聚类**高维**数据
- 对 **Minpts** 的参数非常敏感

以下哪种情况对 Kmeans 模型的影响较小 ()

- A. 给定错误聚类个数
- B. 簇的大小分布不均衡
- C. 不同簇的方差相差较大
- D. 数据有各向异性分布

下列哪些是 K 均值 (Kmeans) 聚类的缺点:

- A. 超参数 “聚类簇数” K不好把握
- B. 对初始簇中心的选取十分敏感
- C. 采用EM算法迭代优化, 聚类收敛速度慢
- D. 算法的可解释性比较差

对于 DBSCAN, 参数 ϵ 固定, 当 MinPts 取值较大时, 会导致

- A. 能很好的区分各类簇
- B. 只有高密度的点的聚集区划为簇, 其余划为噪声
- C. 低密度的点的聚集区划为簇, 其余的划为噪声
- D. 无影响

已知6个点: A(2, 1)、B(3, 5)、C(6, 4)、D(5, 6)、E(7, 3)、F(1, 3), 根据KMeans算法的主要步骤, 假设K=3, 随机选取A(2, 1)、C(6, 4)、E(7, 3) 作为三个初始簇中心, 使用欧氏距离作为相似性判断, 请问正确的聚类结果是 ()

- A. 三个簇: { A, F }; { B, D }; { C, E }
- B. 三个簇: { A, F, B }; { D }; { C, E }
- C. 三个簇: { A, F }; { B, D, C }; { E }
- D. 三个簇: { A, F }; { B, C }; { D, E }



Q&A

TRANSWARP
星环科技